SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1007	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	TARUNIKA S	**Reg. No.:**	192511024
Date:	30/7/2026	**Duration:**	60 minutes

Code of Conduct
I TARUNIKA S (192511024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: TARUNIKA S

Annexure A – Architecture Design Assignment

Smart High-Speed Rail Passenger Experience Platform

1. System Overview and Requirements Analysis

The Smart High-Speed Rail Passenger Experience Platform is a digital ecosystem that supports passengers before, during, and after their journey on a high-speed rail network. It unifies ticketing and booking, real-time train tracking, seat and coach navigation, onboard entertainment, in-journey notifications, digital payments, and post-journey feedback into a single connected platform accessible via mobile app, web portal, in-train kiosks, and station signage.

1.1 Functional Requirements

- Passenger registration, login, and profile management across devices.
- Ticket search, booking, cancellation, and e-ticket generation with QR/barcode validation.
- Live train location, expected arrival time, delay alerts, and platform information.
- Seat/coach navigation, onboard entertainment streaming, and Wi-Fi access management.

1.2 Non-Functional Requirements and Quality Attributes

Quality Attribute	Requirement / Rationale
Scalability	Must handle seasonal spikes in bookings and simultaneous data streams from thousands of onboard sensors and users nationwide.
Performance	Real-time location and delay updates must reach passengers within seconds; ticket booking must respond in under 2 seconds.
Availability	The platform must remain operational 24x7, including during peak travel seasons and partial component failures.
Security	Passenger identity, payment data, and travel records must be protected via encryption, authentication, and access control.
Maintainability	Individual features (e.g., entertainment, payments) must be upgraded independently without affecting the whole system.

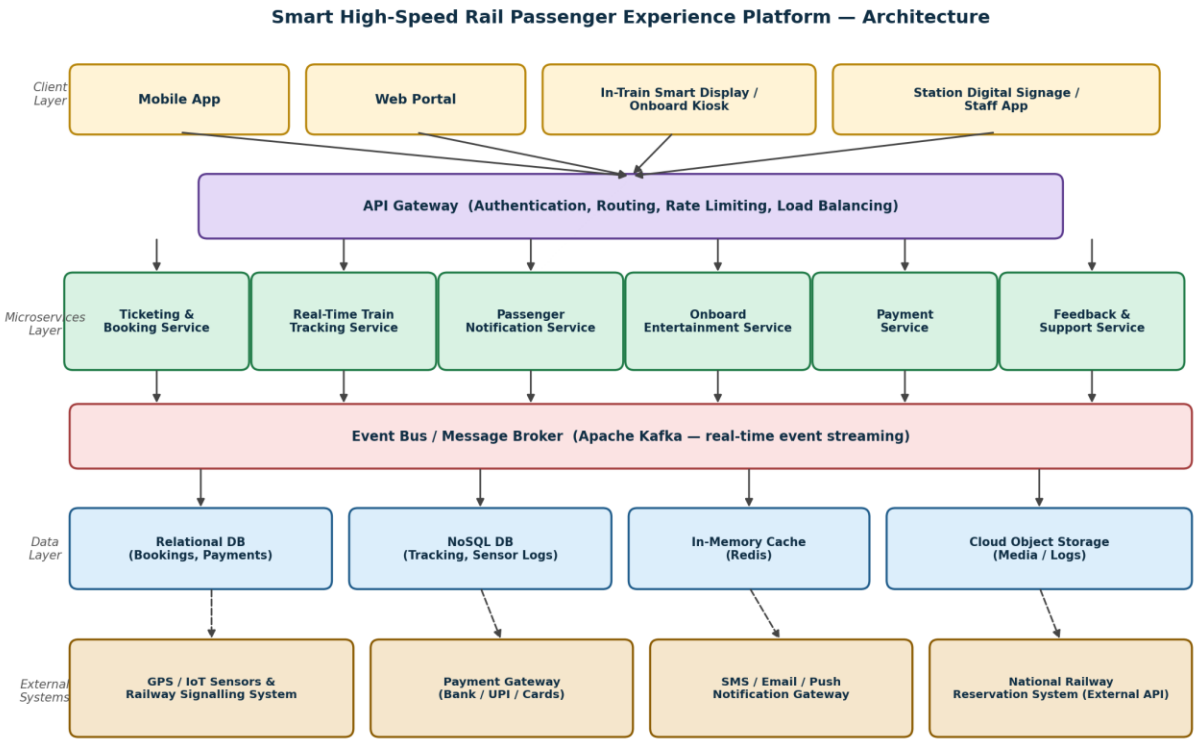
2. Selected Software Architecture

A Hybrid Microservices and Event-Driven Architecture is selected for this platform, combined with an API Gateway as the single entry point for all client applications.

2.1 Justification

- Microservices decompose the platform into independently deployable services (ticketing, tracking, notifications, entertainment, payments, support), enabling teams to scale and update each capability without redeploying the entire system directly supporting maintainability and scalability.
- An event-driven backbone (message broker) is essential because train location, delay alerts, and sensor data are continuously generated streams that must be pushed to many services and passengers in real time, rather than being repeatedly polled.
- The API Gateway centralizes authentication, rate limiting, and request routing for the four client channels (mobile, web, kiosk, staff app), simplifying security enforcement and reducing duplicated logic.

3. Software Architecture Diagram



4. Component Responsibilities and Interactions

Component	Responsibility and Interaction
Client Applications	Mobile app, web portal, in-train display, and staff app send HTTPS requests to the API Gateway and receive real-time push updates.
API Gateway	Authenticates every request, routes it to the correct microservice, applies rate limiting, and load-balances traffic across service instances.
Ticketing & Booking Service	Handles search, booking, cancellation, and e-ticket generation; writes to the relational database and publishes booking events.
Real-Time Tracking Service	Consumes GPS/IoT sensor and signalling data from the event bus and maintains live train position and delay status.
Notification Service	Subscribes to booking, tracking, and payment events, then dispatches SMS/email/push alerts through the notification gateway.

5. Quality Attribute Analysis

- ❖ Scalability: Each microservice scales horizontally and independently (e.g., Tracking Service scales during peak travel hours) without affecting other services.
- ❖ Security: The API Gateway enforces token-based authentication (OAuth2/JWT) and TLS encryption; the Payment Service isolates sensitive data behind a dedicated boundary.
- ❖ Performance: The event bus and Redis cache minimize latency for real-time updates and frequently accessed data such as train status.
- ❖ Reliability: Kafka's durable, replicated event log ensures no delay or safety notification is lost even if a consuming service is temporarily unavailable.
- ❖ Availability: Redundant service instances behind the API Gateway and asynchronous, decoupled communication prevent a single service failure from taking down the whole platform.

6. Architectural Justification, Trade-offs, and Future Scope

The hybrid microservices and event-driven approach was chosen over alternatives such as a monolithic layered architecture or a simple client-server model because passenger-facing rail systems demand continuous real-time data flow, independent scalability of high-traffic features (tracking, notifications), and strict fault isolation between safety-related and non-critical services.

6.1 Trade-offs Considered

- ❖ Increased operational complexity (service orchestration, monitoring, distributed tracing) was accepted in exchange for independent scalability and fault isolation.
- ❖ Eventual consistency between services (via asynchronous events) was accepted over strict immediate consistency, since near-real-time updates are acceptable for tracking and notifications but not for payment confirmation, which uses synchronous calls.

6.2 Future Enhancements

- ➢ New services (e.g., AI-based journey recommendations, predictive delay analytics) can be added by subscribing to existing events without modifying current services.
- ➢ The architecture supports integration with additional external systems, such as smart-city transit APIs, through the API Gateway and event bus.
- ➢ Containerized deployment (Docker/Kubernetes) allows the platform to scale elastically across cloud regions to support long-term passenger growth.